

Atividade Prática - Árvores Binárias

Calculadora de Expressões Matemáticas com Árvore de Expressão

Disciplina: DSA - Estruturas de Dados e Algoritmos

Conteúdos: árvores binárias, nós e referências, pilhas, notação infixa/pós-fixa, recursividade, percursos e complexidade.

Objetivo: desenvolver um programa que receba uma expressão matemática, construa automaticamente uma árvore binária que a represente e utilize a árvore para gerar percursos e calcular o resultado.

Regra principal: o cálculo deve ser realizado a partir da árvore construída. É proibido utilizar `eval()`, `exec()` ou mecanismo equivalente.

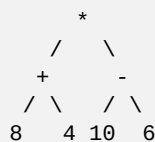
1. O problema

Nesta atividade, a árvore não será montada manualmente. Ela deverá ser criada automaticamente a partir da expressão fornecida pelo usuário.

Exemplo de entrada:

```
((8 + 4) * (10 - 6))
```

Árvore conceitual:



A própria árvore deverá produzir o resultado:

```
(8 + 4) * (10 - 6)
12 * 4
Resultado = 48
```

2. Expressões aceitas

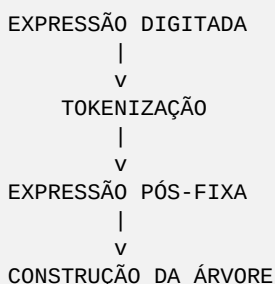
- números inteiros positivos e decimais positivos;
- operadores +, -, * e /;
- parênteses;
- espaços opcionais.

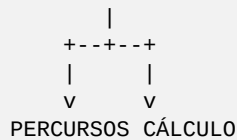
Exemplos:

```
8 + 4
8 + 4 * 2
(8 + 4) * 2
((15 - 3) / 4) + (2 * 5)
((20 / 5) + 3) * (9 - (2 + 1))
```

Escopo: potência, raiz, funções, variáveis e sinal unário negativo são opcionais.

3. Visão geral





Desenvolva e teste uma etapa por vez.

4. Estrutura do nó

Implemente uma classe No com valor, filho esquerdo e filho direito.

```
class No:
    def __init__(self, valor):
        self.valor = valor
        self.esquerda = None
        self.direita = None
```

Números serão folhas; operadores serão nós internos.

5. Tokenização

A expressão precisa ser separada em tokens. O número 15 deve ser um único token, não os caracteres 1 e 5.

```
Entrada:
(15 + 3) * 2

Tokens:
['(', '15', '+', '3', ')', '*', '2']

def tokenizar(expressao):
    # retorne uma lista de tokens
```

A função deverá reconhecer números com vários algarismos e números decimais.

6. Precedência

A expressão $8 + 4 * 2$ representa $8 + (4 * 2)$, e não $(8 + 4) * 2$.

Operadores	Precedência
* e /	maior
+ e -	menor
()	controlam o agrupamento

7. Converter infix para pós-fixa

A entrada está em notação infix:

```
8 + 4 * 2
```

Converta-a para pós-fixa:

```
8 4 2 * +
```

Use uma pilha para operadores e parênteses.

```
def precedencia(operador):
    # devolva a prioridade

def para_posfixa(tokens):
    # devolva a sequência pós-fixa
```

- 1 Números vão diretamente para a saída.
- 2 '(' entra na pilha.
- 3 Ao encontrar ')', retire operadores até o '(' correspondente.

- 4 Ao encontrar operador, respeite a precedência antes de empilhá-lo.
- 5 No final, transfira os operadores restantes para a saída.

8. Construir a árvore

Percorra a pós-fixa usando uma pilha de nós.

- 1 Número: crie um nó e empilhe.
- 2 Operador: retire dois nós.
- 3 O primeiro nó retirado é o filho direito.
- 4 O segundo é o filho esquerdo.
- 5 Crie o nó operador, conecte os filhos e empilhe a nova subárvore.

```
def construir_arvore(posfixa):  
    # devolva a raiz
```

A ordem dos pop () é essencial: inverter os filhos altera subtrações e divisões.

9. Percursos

Implemente recursivamente:

```
def pre_ordem(no):  
    ...  
  
def em_ordem(no):  
    ...  
  
def pos_ordem(no):  
    ...
```

Para $8 + 4 * 2$:

```
Pré-ordem: + 8 * 4 2  
Em ordem: 8 + 4 * 2  
Pós-ordem: 8 4 2 * +
```

Explique em comentário a relação entre os percursos e as notações prefixa, infixa e pós-fixa.

10. Reconstruir a expressão

Gere novamente uma expressão infixa com parênteses percorrendo a árvore. Não vale imprimir a entrada original.

```
def gerar_expressao(no):  
    # Exemplo:  
    # ((8 + 4) * (10 - 6))
```

11. Calcular pela árvore

```
def calcular(no):  
    # devolva o resultado da subárvore
```

- 1 Se o nó for folha, devolva o número.
- 2 Calcule recursivamente a esquerda.
- 3 Calcule recursivamente a direita.
- 4 Aplique o operador do nó aos resultados.

Trate divisão por zero de forma controlada.

12. Saída do programa

- tokens reconhecidos;

- expressão pós-fixa;
- pré-ordem;
- em ordem;
- pós-ordem;
- expressão reconstruída;
- resultado calculado pela árvore.

Exemplo conceitual:

```
Digite uma expressão:
((20 / 5) + 3) * (9 - (2 + 1))

Pós-fixa:
20 5 / 3 + 9 2 1 + - *

Pré-ordem:
* + / 20 5 3 - 9 + 2 1

Pós-ordem:
20 5 / 3 + 9 2 1 + - *

Expressão reconstruída:
(((20 / 5) + 3) * (9 - (2 + 1)))

Resultado:
42
```

13. Testes obrigatórios

Teste	Expressão	Resultado
1	$8 + 4 * 2$	16
2	$(8 + 4) * 2$	24
3	$((15 - 3) / 4) + (2 * 5)$	13
4	$((20 / 5) + 3) * (9 - (2 + 1))$	42
5	$12.5 + 2.5 * 4$	22.5

14. Validação mínima

- divisão por zero;
- parênteses incompatíveis;
- expressão vazia;
- caractere não aceito.

Não é necessário criar um compilador completo; a validação deve impedir erros evidentes e resultados silenciosamente incorretos.

15. Questões de análise

- 1 Por que a expressão precisa ser tokenizada?
- 2 Por que uma pilha é adequada para operadores e parênteses?
- 3 Por que o primeiro pop na construção da árvore corresponde ao filho direito?
- 4 Qual a relação entre pós-ordem e notação pós-fixa?
- 5 Para uma árvore com n nós, qual a complexidade de tempo para calcular toda a expressão? Justifique.

6 O que determina a quantidade máxima de chamadas recursivas simultâneas?

Responda em comentários ao final do arquivo.

16. Desafio opcional

- suportar números negativos;
- adicionar potência (^);
- mostrar a árvore hierarquicamente no terminal;
- contar nós, folhas e altura;
- permitir novas expressões sem reiniciar o programa.

17. Critérios de avaliação

Critério	Valor
Tokenização e entrada	1,5
Infixa -> pós-fixa	2,0
Construção da árvore	2,0
Percursos	1,0
Reconstrução da expressão	1,0
Cálculo recursivo	1,5
Organização, validação e análise	1,0
Total	10,0

18. Entrega

Entregue um único arquivo .py, identificado conforme orientação do professor, contendo programa completo, testes e respostas da análise em comentários.

A construção da árvore binária é obrigatória. Soluções que apenas calcularem diretamente a expressão não atendem ao objetivo.